

A BEGINNER'S GUIDE TO WRITING SQL IN MICROSOFT ACCESS

Student Course Registration Edition

What SQL Does

Data Definition Language (DDL) creates or changes database objects. **Data Manipulation Language (DML)** inserts, retrieves, updates and deletes records.

Open SQL View

- Choose **Create > Query Design** and close the Show Table or Add Tables window.
- Choose **View > SQL View**.
- Run one statement at a time. Clear SQL View before entering the next statement.

Access action and DDL queries can change the database immediately. Use a working copy of your Lab 9 database, keep a clean backup, and preview affected rows before UPDATE or DELETE.

Clause Order for a SELECT Query

```
SELECT field_list
FROM table_expression
WHERE row_criteria
GROUP BY grouping_fields
HAVING group_criteria
ORDER BY sort_fields;
```

Only SELECT and FROM are required in this pattern. Add the other clauses when the task needs them.

The Practice Database Used in This Guide

The examples use the tables and data that students construct step by step in Lab 9. The compact dataset makes every expected result reproducible.

Table	Primary key	Important foreign keys	Rows
Students	StudentID	-	4
Instructors	InstructorID	-	3
Courses	CourseID	InstructorID, PreReqCourseID	5
Registrations	RegistrationID	StudentID, CourseID	7

Use Explicit Access Data Types

Access SQL type	Typical use	Important note
COUNTER	AutoNumber primary key	Generated by Access
LONG	Foreign key to COUNTER	Compatible with AutoNumber
TEXT(n)	Short text	State a suitable length
DATETIME	Dates and times	Date literals use #...#
BYTE	Small whole numbers	Suitable for course credits
DOUBLE	Decimal numbers	Suitable for GPA
BIT	Yes/No	Use True or False

Avoid vague generic types when teaching Access. In particular, NUMBER can map differently from an intended integer key. Use LONG for foreign keys and DOUBLE for decimal values.

This guide does not place CHECK or DEFAULT clauses inside CREATE TABLE statements. Apply the required Validation Rule and Default Value through Access Table Design, following Lab 7.

Names and Aliases

- Use stable object names without spaces where practical.
- Place square brackets around an output alias containing spaces, for example [Weekly Study Hours].
- SQL keywords are not case-sensitive, but uppercase keywords make code easier to scan.

DDL: Create the Parent Tables First

Create tables in dependency order. Instructors and Students do not depend on another table.

```
CREATE TABLE Instructors (  
    InstructorID COUNTER CONSTRAINT PK_Instructors PRIMARY KEY,  
    FirstName TEXT(50) NOT NULL,  
    LastName TEXT(50) NOT NULL,  
    Department TEXT(50) NOT NULL  
);
```

```
CREATE TABLE Students (  
    StudentID COUNTER CONSTRAINT PK_Students PRIMARY KEY,  
    FirstName TEXT(50) NOT NULL,  
    LastName TEXT(50) NOT NULL,  
    Email TEXT(100) NOT NULL,  
    DateOfBirth DATETIME,  
    Gender TEXT(20),  
    IsActive BIT NOT NULL,  
    GPA DOUBLE,  
    CONSTRAINT UQ_Students_Email UNIQUE (Email)  
);
```

PRIMARY KEY provides uniqueness and disallows Null. The named **UNIQUE** constraint prevents two students from sharing the same email address.

DDL: Create Related Tables

A foreign key may be **required** (NOT NULL) or **optional** (nullable). Being a foreign key does not automatically make a field nullable.

```
CREATE TABLE Courses (  
    CourseID COUNTER CONSTRAINT PK_Courses PRIMARY KEY,  
    CourseCode TEXT(10) NOT NULL,  
    CourseName TEXT(100) NOT NULL,  
    Credits BYTE NOT NULL,  
    Department TEXT(50) NOT NULL,  
    IsElective BIT NOT NULL,  
    InstructorID LONG NOT NULL,  
    PreReqCourseID LONG,  
    CONSTRAINT UQ_Courses_Code UNIQUE (CourseCode),  
    CONSTRAINT FK_Courses_Instructors FOREIGN KEY (InstructorID)  
        REFERENCES Instructors (InstructorID)  
);
```

```
CREATE TABLE Registrations (  
    RegistrationID COUNTER CONSTRAINT PK_Registrations PRIMARY KEY,  
    StudentID LONG NOT NULL,  
    CourseID LONG NOT NULL,  
    RegistrationDate DATETIME NOT NULL,  
    Semester TEXT(20) NOT NULL,  
    Grade TEXT(2),  
    CONSTRAINT FK_Registrations_Students FOREIGN KEY (StudentID)  
        REFERENCES Students (StudentID),  
    CONSTRAINT FK_Registrations_Courses FOREIGN KEY (CourseID)  
        REFERENCES Courses (CourseID),  
    CONSTRAINT UQ_Registrations UNIQUE (StudentID, CourseID, Semester)  
);
```

The three-field unique constraint prevents the same student from registering for the same course twice in one semester.

ALTER TABLE and Safe DROP Practice

Use a disposable table when practising structural changes. Never substitute a teaching table into the final DROP TABLE statement.

```
CREATE TABLE DDLPractice (  
    PracticeID COUNTER CONSTRAINT PK_DDLPractice PRIMARY KEY,  
    PracticeName TEXT(50) NOT NULL,  
    PracticeDate DATETIME,  
    IsComplete BIT,  
    Score DOUBLE  
);
```

Add, modify and remove a field

```
ALTER TABLE DDLPractice  
ADD COLUMN Notes TEXT(100);
```

```
ALTER TABLE DDLPractice  
ALTER COLUMN Notes TEXT(150);
```

```
ALTER TABLE DDLPractice  
ADD COLUMN TemporaryFlag BIT;
```

```
ALTER TABLE DDLPractice  
DROP COLUMN TemporaryFlag;
```

Add a named unique constraint

```
ALTER TABLE DDLPractice  
ADD CONSTRAINT UQ_DDLPractice_Name UNIQUE (PracticeName);
```

DROP TABLE removes both the data and the table definition. Use DROP TABLE DDLPractice; only after completing the disposable exercise.

INSERT INTO: Add One Record at a Time

In Access, a VALUES append query adds one record. Use another INSERT INTO statement for each additional record, or use INSERT INTO . . . SELECT to append a result set.

```
INSERT INTO DDLPractice
(PracticeName, Notes, PracticeDate, IsComplete, Score)
VALUES
('First row', 'Created with Access SQL', #2026-10-01#, True, 3.5);
```

For tables with an AutoNumber key, omit the AutoNumber field when Access should generate the next identifier.

Text values use single quotes. Access date literals use hash marks. Yes/No values use True or False.

Field Lists Prevent Mistakes

- Name the destination fields explicitly.
- Keep the field order aligned with the value order.
- Do not supply an AutoNumber value unless retaining a known identifier is intentional.
- Check required fields, unique constraints and foreign keys before running the append query.

SELECT: Retrieve, Rename and Calculate

Retrieve selected fields

```
SELECT CourseCode, CourseName, Credits
FROM Courses
ORDER BY CourseCode;
```

Expected result: 5 rows.

Retrieve every field

```
SELECT *
FROM Courses;
```

Use * for quick exploration. Name the fields explicitly in saved teaching and reporting queries so the output remains predictable.

Aliases and computed columns

```
SELECT CourseCode, CourseName,
       Credits * 3 AS [Weekly Study Hours]
FROM Courses
ORDER BY CourseCode;
```

Expected result: 5 rows. ICT550 produces 12 weekly study hours; each 3-credit course produces 9.

WHERE: Filter Rows

WHERE filters individual rows before grouping. Combine conditions with AND, OR and NOT; use parentheses when mixing operators.

```
SELECT StudentID, FirstName, LastName, GPA
FROM Students
WHERE IsActive = True AND GPA >= 3.5
ORDER BY GPA DESC;
```

Expected result: 2 rows - Chen Mei and Aina.

```
SELECT StudentID, FirstName, LastName, IsActive, GPA
FROM Students
WHERE IsActive = False OR NOT (GPA >= 3.5)
ORDER BY StudentID;
```

Expected result: 2 rows - Bala and Daniel. Parentheses make the condition controlled by NOT explicit.

BETWEEN and Date Criteria

```
SELECT StudentID, FirstName, LastName, GPA
FROM Students
WHERE GPA BETWEEN 3.5 AND 4
ORDER BY GPA DESC;
```

Expected result: 2 rows. BETWEEN includes both endpoints.

```
SELECT RegistrationID, StudentID, RegistrationDate
FROM Registrations
WHERE RegistrationDate >= #2026-10-01#
  AND RegistrationDate < #2026-11-01#
ORDER BY RegistrationDate;
```

Expected result: 7 rows. A start-inclusive/end-exclusive range works reliably for a whole month, including values that contain a time.

IN, LIKE, IS NULL and DISTINCT

IN replaces repeated OR comparisons

```
SELECT InstructorID, FirstName, LastName, Department
FROM Instructors
WHERE Department IN ('Information Systems', 'Data Science')
ORDER BY Department, LastName;
```

Expected result: 2 rows.

LIKE uses the database's wildcard mode

A normal .accdb uses **ANSI-89** by default: * means zero or more characters and ? means one character. If the database is explicitly set to ANSI-92, use % and _ instead.

```
SELECT CourseCode, CourseName
FROM Courses
WHERE CourseName LIKE '*Data*'
ORDER BY CourseCode;
```

Expected result: 4 rows.

```
SELECT CourseCode, CourseName
FROM Courses
WHERE CourseCode LIKE 'ICT4?0';
```

Expected result: 1 row - ICT450.

IS NULL and DISTINCT

```
SELECT CourseCode, CourseName
FROM Courses
WHERE PreReqCourseID IS NULL;
```

```
SELECT DISTINCT Department
FROM Courses
ORDER BY Department;
```

Expected results: 1 course without a prerequisite and 3 distinct departments.

Aggregate Functions

Aggregate functions summarise several rows. `COUNT ( field )` ignores Null values in that field; `COUNT ( * )` counts rows.

```
SELECT COUNT(StudentID) AS StudentCount,  
       AVG(GPA) AS AverageGPA,  
       MAX(GPA) AS HighestGPA,  
       MIN(GPA) AS LowestGPA  
FROM Students;
```

Result	Expected value
StudentCount	4
AverageGPA	3.4475
HighestGPA	3.88
LowestGPA	2.94

SUM

```
SELECT SUM(Credits) AS TotalCredits  
FROM Courses;
```

Expected result: 16 total credits.

GROUP BY and HAVING

GROUP BY forms groups. WHERE filters rows before grouping; HAVING filters the completed groups.

```
SELECT StudentID,  
       COUNT(RegistrationID) AS RegistrationCount  
FROM Registrations  
GROUP BY StudentID  
ORDER BY StudentID;
```

Expected result: 4 groups. StudentID 4 has one registration; the others have two.

```
SELECT StudentID,  
       COUNT(RegistrationID) AS RegistrationCount  
FROM Registrations  
GROUP BY StudentID  
HAVING COUNT(RegistrationID) > 1  
ORDER BY StudentID;
```

Expected result: 3 groups - StudentID 1, 2 and 3.

Every selected field that is not inside an aggregate function must appear in the GROUP BY clause.

INNER JOIN: Combine Related Tables

Use explicit `INNER JOIN ... ON` syntax so the relationship path is visible. Joined numeric fields must use compatible types; a COUNTER primary key is paired with a LONG foreign key.

Join two tables

```
SELECT R.RegistrationID, R.StudentID,  
       C.CourseCode, C.CourseName  
FROM Registrations AS R  
INNER JOIN Courses AS C  
    ON R.CourseID = C.CourseID  
ORDER BY R.RegistrationID;
```

Expected result: 7 rows.

Join three tables

```
SELECT S.StudentID,  
       S.FirstName & ' ' & S.LastName AS StudentName,  
       C.CourseCode, C.CourseName, R.Semester, R.Grade  
FROM (Students AS S  
INNER JOIN Registrations AS R  
    ON S.StudentID = R.StudentID)  
INNER JOIN Courses AS C  
    ON R.CourseID = C.CourseID  
ORDER BY S.StudentID, C.CourseCode;
```

Expected result: 7 rows. Access requires parentheses around this nested join path.

A Four-Table Join

```
SELECT S.FirstName & ' ' & S.LastName AS StudentName,  
       C.CourseCode, C.CourseName,  
       I.FirstName & ' ' & I.LastName AS InstructorName,  
       R.Semester, R.Grade  
FROM ((Students AS S  
INNER JOIN Registrations AS R  
      ON S.StudentID = R.StudentID)  
INNER JOIN Courses AS C  
      ON R.CourseID = C.CourseID)  
INNER JOIN Instructors AS I  
      ON C.InstructorID = I.InstructorID  
ORDER BY S.StudentID, C.CourseCode;
```

Expected result: 7 rows, one for each registration.

Why Explicit Joins Are Preferred

- The key relationship is kept in the ON clause.
- Filtering criteria remain in the WHERE clause.
- A missing join condition cannot silently become a Cartesian product as easily.
- The query matches the relationship structure taught in the ERD and Access Relationships window.

Subqueries

A subquery is a SELECT statement inside another statement. The inner query supplies a value or set used by the outer query. An empty subquery result is an empty set, not automatically an error.

Use IN when the inner query can return several values

```
SELECT StudentID, FirstName, LastName
FROM Students
WHERE StudentID IN (
    SELECT StudentID
    FROM Registrations
    WHERE CourseID = (
        SELECT CourseID
        FROM Courses
        WHERE CourseCode = 'ICT450'
    )
)
ORDER BY StudentID;
```

Expected result: 3 students - StudentID 1, 2 and 3.

Use = only when one value is guaranteed

```
SELECT StudentID, FirstName, LastName, GPA
FROM Students
WHERE GPA >= (
    SELECT AVG(GPA)
    FROM Students
)
ORDER BY GPA DESC;
```

Expected result: 2 rows. The inner aggregate returns one value: 3.4475.

Do not use = with a subquery that can return several rows. Use IN, or rewrite the query so one value is guaranteed.

UPDATE and DELETE Safely

Before an action query, run a SELECT with the same WHERE criteria. Continue only when the preview returns exactly the intended rows.

Preview and perform a reversible UPDATE

```
SELECT StudentID, FirstName, LastName, GPA
FROM Students
WHERE StudentID = 4 AND GPA = 2.94;
```

```
UPDATE Students
SET GPA = 3.04
WHERE StudentID = 4 AND GPA = 2.94;
```

```
UPDATE Students
SET GPA = 2.94
WHERE StudentID = 4 AND GPA = 3.04;
```

Each UPDATE affects one row. The final statement restores the original practice value.

Preview DELETE on a disposable table

```
SELECT *
FROM DDLPractice
WHERE PracticeName = 'First row';
```

```
DELETE FROM DDLPractice
WHERE PracticeName = 'First row';
```

An UPDATE or DELETE without a suitable WHERE clause can affect every row. A delete query is not undoable; keep a backup.

Common Access SQL Errors

Incorrect or unsafe idea	Correct Access practice
A foreign key cannot be NOT NULL	Optionality is separate; a required FK may be NOT NULL
Use _ for one unknown character	ANSI-89 Access uses ?; ANSI-92 uses _
Put dates in quotes	Use #date# literals
Use LIKE on a date	Use an inclusive/exclusive date range
Join tables with commas	Use explicit INNER JOIN ... ON
A no-row subquery is a NULL	It returns an empty result set
Paste comments with the SQL	Run only the executable statement in SQL View

Quick Reading Checklist

- Which table supplies each selected field?
- Which rows are filtered by WHERE?
- Are groups filtered by HAVING only after GROUP BY?
- Does each join connect a primary key to the correct foreign key?
- Can a subquery return one value or several values?
- Does an action query include a preview, backup or reversal?